OPERATING SYSTEMS – ASSIGNMENT 1

CO1: Operating-system types, structures, services and system calls

Name: _____ Register Number: _____

Course: CSA04 – Operating Systems

Note: The questions below are restated from the supplied assignment sheet, followed by model answers. Replace measured results and execution screenshots with your own Linux/GCC results before submission.

TASK 1 – Requirements of Each Computing Environment

ASSIGNMENT QUESTION

Evaluate the OS requirements of bedside monitors, handheld tablets, desktop workstations, database server, imaging server and partner-hospital link. Rank response time, reliability, throughput, battery and security needs; identify conflicts, patient-safety priorities and relevant OS-history lessons. Compare this with using one general-purpose OS everywhere.

AIM

To understand requirements of each computing environment and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

The hospital should use a specialised OS for each environment rather than forcing one general-purpose OS onto every device.

Ranked requirements:

TOPIC	EXPLANATION
Bedside monitors	(1) deterministic response time, because an abnormal reading must trigger an alarm within 10 ms; (2) reliability and availability; (3) device and patient-data security; (4) low memory and power use; (5) throughput. A compact hard real-time embedded OS is appropriate.
Nurses' tablets	(1) battery efficiency; (2) security and authentication; (3) reliable wireless communication; (4) responsive user interface;

	(5) moderate multitasking. A handheld/mobile OS is suitable.
Doctors' workstations	(1) usability and responsiveness; (2) security and privacy; (3) compatibility with clinical applications; (4) multitasking and I/O throughput. A desktop time-sharing OS is suitable.
Patient-record database	(1) continuous availability and data integrity; (2) security and access control; (3) transaction throughput; (4) recoverability; (5) response time. A server OS with clustering and fail-over is required.
MRI imaging server	(1) high throughput and parallel processing; (2) memory and storage bandwidth; (3) reliability; (4) security. A multiprocessor server OS is suitable.
Partner-hospital connection	(1) confidentiality and integrity; (2) availability; (3) interoperability and network performance. A distributed-system design with encrypted communication is required.

Patient safety is most directly affected by monitor alarm deadlines and database availability/integrity. Conflicts include deterministic low-latency scheduling versus high overall throughput, and security checks versus response time. Batch systems taught efficient job processing, time-sharing supports many interactive users, personal-computer systems support individual workstations, and mobile systems emphasise power management.

If one general-purpose OS were used everywhere, scheduling would not guarantee the 10 ms alarm deadline: worst-case delay could be unbounded under overload. It would also consume excessive memory/storage on monitors and battery on tablets, while increasing administration complexity because every device would carry unnecessary services. A specialised design gives predictable timing and a smaller footprint. Complexity: the monitor's alarm decision is O(1) per sensor event; a general-purpose scheduler may have workload-dependent delay, so it cannot provide the required hard deadline.

TASK 2 – Types of Operating Systems

ASSIGNMENT QUESTION

Compare mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard and soft) and handheld OS types. Select the best type for each hospital environment; compare symmetric and asymmetric multiprocessing for MRI; evaluate a fail-over cluster against one large multiprocessor database machine.

AIM

To understand types of operating systems and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

Recommended choices:

TOPIC	EXPLANATION
Monitors	hard real-time embedded OS. A missed 10 ms alarm deadline is unacceptable.
Tablets	handheld/mobile OS with power management, wireless networking and secure device management.
Workstations	desktop time-sharing OS for interactive clinical applications.
Database	clustered server OS with shared or replicated storage and automatic fail-over. A mainframe-class system is another possible high-reliability option, but clustering avoids one machine being a single point of failure.
Imaging	multiprocessor OS, preferably symmetric multiprocessing (SMP). In SMP, processors share memory and the OS schedules work across them, which suits parallel image-processing tasks. Asymmetric multiprocessing assigns specialised work to particular processors and may be useful for dedicated acquisition or control tasks, but is less flexible for general parallel workloads.

Partner link	distributed-system architecture, because independent hospital systems exchange records over a network.

Hard real-time systems guarantee deadlines; soft real-time systems aim for good average response but tolerate occasional delays. MRI image enhancement can often be soft real-time, while monitor alarms require hard real-time behaviour.

A single large multiprocessor database server can provide high throughput, but the entire machine remains a single point of failure. A cluster of smaller servers can fail over within seconds and continue service, at the cost of extra hardware, replication and administration. With p processors, ideal parallel throughput can approach O(p) improvement, but contention and serial work reduce the gain. Memory capacity is limited to the machine's installed RAM in a single-server design; a cluster can distribute memory and data across nodes, although coordination is required.

TASK 3 – Operating-System Structures

ASSIGNMENT QUESTION

Evaluate simple/monolithic, layered, microkernel, modular and hybrid structures for reliability, performance and updating. Choose structures for monitors and database server; describe Linux, Windows, Android and FreeRTOS/QNX. Compare microkernel and monolithic monitor designs.

AIM

To understand operating-system structures and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

A monolithic kernel places most OS services in kernel space. It can be fast because components communicate directly, but a faulty driver can crash the whole system and the kernel is harder to isolate. A layered OS divides services into levels, improving organisation and testing but potentially adding overhead between layers. A microkernel keeps only essential functions in kernel space and moves drivers/services to user space; this improves fault isolation and maintainability, though message passing can add latency. A modular kernel supports loadable modules, balancing performance and extensibility. Hybrid kernels combine approaches.

For bedside monitors, choose a small microkernel or safety-oriented real-time kernel. Isolation of drivers and predictable scheduling are more important than maximum general-purpose throughput. For the database server, choose a modular server kernel such as Linux, with mature storage, networking, security and monitoring support.

Linux is monolithic but modular; Windows uses a hybrid design; Android uses the Linux kernel with an application framework and hardware abstraction layers; FreeRTOS is a small real-time kernel, while QNX uses a microkernel architecture.

In a monolithic monitor kernel, a service request may require one system-call transition and direct in-kernel work. In a microkernel, a request may require message passing between client and service processes, adding context switches and latency. However, the microkernel can keep the kernel smaller and isolate a faulty driver. The extra communication cost must be measured against the 10 ms deadline; a properly engineered real-time microkernel can still meet it.

TASK 4 – Operating-System Services

ASSIGNMENT QUESTION

Evaluate OS services for each environment: UI, execution, I/O, file systems, communication, error detection, allocation, accounting, protection and security. Identify services removable from monitors and essential to databases. Compare a reduced handheld OS with a full desktop OS on tablets.

AIM

To understand operating-system services and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

Service selection:

TOPIC	EXPLANATION
Monitor	minimal program execution, sensor I/O, interrupt handling, timing, watchdog/error detection, secure update and access protection. Remove full desktop UI, general-purpose printing, unnecessary background services and broad file-management features.

Tablet	touch UI, application execution, wireless communication, local encrypted storage, power management, authentication and device management.
Workstation	full UI, multitasking, file and network services, printing, application support, authentication, auditing and endpoint protection.
Database	process and memory management, reliable file systems, storage I/O, networking, transaction support, error recovery, resource allocation, auditing, access control, encryption and backup services.
Imaging server	high-throughput I/O, large-file storage, parallel execution, memory management, networking and security.
Inter-hospital link	network communication, encryption, authentication, logging and error detection.

The database must enforce role-based access control, least privilege, audit logging and encryption. Each query should be authorised so one patient's records cannot be read by an unauthorised user or process.

A full desktop OS on tablets adds services, background processes and storage use. This increases RAM and storage requirements, startup time and energy consumption. A handheld OS with only required services improves battery life and reduces attack surface. The space cost is proportional to installed services and their data; startup time and power use rise as more services initialise and remain active.

TASK 5 – System Calls

ASSIGNMENT QUESTION

Explain system-call categories and the steps in read(): library call, trap, mode switch, kernel work and return. Explain parameter passing and strace. Compare direct open/read/write calls with fopen/fread/fwrite buffering.

AIM

To understand system calls and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

System-call categories include process control (fork, exec, wait), file management (open, read, write, close), device management (ioctl), information maintenance (getpid, time), communication (pipe, socket, shared-memory operations) and protection (chmod, access).

TOPIC	EXPLANATION
For read(fd, buffer, count)	the application calls the C library wrapper; the wrapper places the system-call number and arguments in registers or another agreed parameter area; a trap instruction switches the CPU from user mode to kernel mode; the kernel validates the descriptor and buffer, performs or waits for I/O, copies data as required, and returns a byte count or error; the CPU returns to user mode. Parameters can be passed in registers, a memory block referenced by a register, or the stack.
strace records system calls and signals, for example	`strace -c ./program` gives call counts and time summaries.

Direct system calls give explicit control but may cause many transitions. Buffered stdio functions (`fopen`, `fread`, `fwrite`) keep a user-space buffer and combine many small operations into larger system calls. For n bytes and a buffer b, calls are approximately O(n/b); a 4096-byte buffer needs about ceil(n/4096) reads plus an end-of-file read in a simple loop. The library consumes O(b) extra memory but usually reduces system-call overhead.

TASK 6 – Cost of System Calls

ASSIGNMENT QUESTION

Evaluate why system calls cost more than ordinary function calls, how buffer size changes read/write counts, how library buffering helps, and recommend a backup buffer size. Complete the file-copy TODO and compare with mmap.

AIM

To understand cost of system calls and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

A normal function call usually changes control flow within user mode. A system call additionally performs a controlled privilege transition, validates arguments and permissions, may switch address spaces or block for I/O, and returns through the kernel. Therefore it is more expensive.

For a 10,000-byte file, the supplied program expects:

TOPIC	EXPLANATION
1-byte buffer	10,001 read calls (10,000 data reads plus EOF).
512-byte buffer	21 calls (20 data reads plus EOF).
4096-byte buffer	4 calls (three data reads plus EOF).

A 4096-byte or larger measured buffer is a sensible starting point for backups; benchmark 4 KB, 16 KB, 64 KB and 256 KB on the actual storage. Larger buffers reduce call count but consume more memory and eventually give diminishing returns.

Completed TODO 1:

```
while ((n = read(in, buf, bufsize)) > 0) {
   calls++;
   size_t off = 0;
   while (off < (size_t)n) {
      ssize_t w = write(out, buf + off, (size_t)n - off);
      if (w < 0) { close(in); close(out); return -1; }
      off += (size_t)w;
   }
}
if (n < 0) { close(in); close(out); return -1; }
```

For very large files, mmap maps file pages into virtual memory and avoids repeated read() calls. It can reduce explicit copying and system-call frequency, but page faults still occur and the mapped virtual address range must be managed. Mapping is O(1) setup calls plus

page-fault work proportional to pages touched; read/write buffering uses O(n/b) calls and O(b) buffer space.

TASK 7 – Process Control System Calls

ASSIGNMENT QUESTION

Explain fork(), exec(), wait(), exit(), shell command execution, exit status, zombie and orphan processes, and why fork()+exec() is used. Complete the supplied fork/exec/waitpid program and compare posix_spawn or threads.

AIM

To understand process control system calls and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

fork() creates a child process. The child initially has a copy-on-write view of the parent's address space. exec() replaces the calling process image with a new program. wait()/waitpid() lets the parent collect the child's termination status. exit() terminates a process and records its status.

A shell parses a command, forks, the child calls exec to run it, and the parent waits. The parent obtains the result using WIFEXITED and WEXITSTATUS. A zombie has finished but remains in the process table until its parent collects its status. An orphan's parent exits first; it is adopted by a system process that later reaps it. fork()+exec() separates process creation from program selection and allows the shell to set up redirection and pipes before exec.

Completed TODO:

```
pid_t pid = fork();
int status;
if (pid < 0) { perror("fork"); return 1; }
if (pid == 0) {
   execlp("expr", "expr", "6", "*", "7", (char *)NULL);
   _exit(127);
}
if (waitpid(pid, &status, 0) < 0) { perror("waitpid"); return 1; }
printf("job 1 exit status %d\n", WIFEXITED(status) ? WEXITSTATUS(status) : -1);
```

Flushing stdout before fork prevents buffered text from being duplicated in both parent and child. If the buffer is not flushed and the child calls exit(), it flushes its inherited copy too, so redirected output can contain the line twice. _exit() avoids flushing inherited stdio buffers. posix_spawn() can reduce setup overhead; threads use less memory but provide weaker isolation because a faulty command can affect shared process state.

TASK 8 – CPU States and Protection

ASSIGNMENT QUESTION

Explain user/kernel mode, privileged instructions, mode bit, interrupts, traps and timer interrupts. Evaluate the risk if monitor software could disable interrupts. Compare interrupt-driven sensor handling with polling.

AIM

To understand cpu states and protection and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

The CPU runs applications in user mode and the OS in kernel mode. A mode bit indicates the privilege level. Privileged instructions, such as disabling interrupts or changing memory-management settings, are allowed only in kernel mode. A system call or exception causes a controlled trap into the kernel; a hardware device raises an interrupt. A timer interrupt lets the OS regain control and schedule another process.

If any monitor application could disable interrupts, it could prevent sensor events and timer scheduling from being handled. This could delay alarms and compromise patient safety. Memory protection uses virtual address spaces and hardware page permissions to prevent one process from reading or writing another process's memory without authorisation.

Polling checks the sensor at fixed intervals T, so worst-case detection delay is approximately T plus processing time. Interrupt-driven handling responds after interrupt latency, generally avoiding repeated checks. Polling can waste CPU and battery when nothing changes. For a 10 ms deadline, interrupt-driven sensing with bounded interrupt latency is preferred; polling is acceptable only if the interval and worst-case processing time are proven to fit the deadline.

TASK 9 – Memory Hierarchy and I/O Systems

ASSIGNMENT QUESTION

Describe registers, cache, RAM, SSD, hard disk and network storage; place MRI images, records and monitor readings; compare programmed I/O, interrupt-driven I/O and DMA. Evaluate interrupt-per-word transfer versus DMA for a 500 MB image.

AIM

To understand memory hierarchy and i/o systems and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

TOPIC	EXPLANATION
The hierarchy from fastest/smallest to slowest/largest is	CPU registers, cache, main memory (RAM), SSD, hard disk and network storage. Typical approximate access times vary by hardware: registers are sub-nanosecond to a few ns, cache about 1–20 ns, RAM tens to around 100 ns, SSD tens to hundreds of microseconds, HDD several milliseconds, and network storage depends on network and server latency.

MRI images should be stored on high-capacity, reliable imaging storage with a cache for active studies. Patient records belong in replicated database storage with backups. Recent monitor readings should be buffered locally for immediate use and forwarded to a secure central time-series store.

Programmed I/O makes the CPU repeatedly check device status; interrupt-driven I/O lets the device notify the CPU; DMA transfers blocks between a device and memory with limited CPU involvement. DMA is best for large MRI images.

A 500 MB image contains 500 million bytes. Interrupt-per-word creates an enormous number of interrupts (for 4-byte words, about 125 million interrupts), consuming substantial CPU time. DMA can transfer a large block with a small number of completion interrupts. DMA needs appropriately sized buffers and careful ownership/coherency management, but is far more efficient for bulk imaging.

TASK 10 – Measure and Compare

ASSIGNMENT QUESTION

Measure copy-program system calls with strace, user/kernel time, creation time for 1,000 processes and threads, and compare the measurements with expectations.

AIM

To understand measure and compare and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

TOPIC	EXPLANATION
Expected trends	increasing the copy buffer reduces read/write system-call counts approximately in proportion to 1/b. `strace -c ./copy` reports actual calls; `/usr/bin/time -v ./program` reports elapsed time and resource information. Process creation generally costs more memory and time than thread creation because processes have separate address spaces and more kernel bookkeeping, while threads share the process address space and resources.

A reproducible measurement procedure is:

1. Compile with `gcc -O2 -Wall program.c -o program` (add `-pthread` for thread programs).

2. Run `strace -c ./copy` for each buffer size.

3. Run `/usr/bin/time -v ./copy`.

4. Time loops creating and joining 1,000 processes, then 1,000 threads.

5. Repeat several times and report the median.

TOPIC	EXPLANATION
Do not invent machine timings	they depend on CPU, kernel, compiler, storage and load. Record your own results in a table. Expected complexity: creating n processes or

	threads is O(n) total; process memory overhead is larger because of separate address-space metadata, while thread stacks and thread-control structures are smaller and shared process memory is O(1) additional address-space duplication.
Alternative mmap	it can reduce explicit read/write calls, but mapped pages still consume virtual address space and resident memory as touched. For a very large file, map in windows rather than mapping the entire file if address-space or memory pressure is a concern.

TASK 11 – Recommend and Defend the Operating-System Plan

ASSIGNMENT QUESTION

Recommend the complete OS plan for the hospital, including type, structure, services, protection and I/O for each environment; risks; support for three hospitals; and compare with placing everything in VMs on two large servers.

AIM

To understand recommend and defend the operating-system plan and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

Final plan:

TOPIC	EXPLANATION
Bedside monitors	dedicated hard real-time embedded OS, small kernel, interrupt-driven sensor input, watchdog and signed updates.
Tablets	managed handheld OS with encryption, authentication, remote lock/wipe and power management.
Workstations	supported desktop OS with endpoint protection and role-based access.

Database	clustered server OS with replicated storage, automatic fail-over, backups, transaction logging and strict access control.
MRI	multiprocessor server OS using SMP, DMA and high-throughput storage.
Partner hospitals	distributed architecture using authenticated, encrypted APIs and replicated records with audit trails.

Risks include configuration complexity, network outages, software vulnerabilities, replication errors and fail-over misconfiguration. Regular testing, backups, monitoring and disaster-recovery exercises are required. The plan scales to three hospitals by using secure distributed services and consistent identity/access policies.

Moving every workload into VMs on two large servers simplifies hardware management but creates contention and a large failure domain. Virtualisation adds CPU and memory overhead and may introduce scheduling jitter. It is unsuitable for the bedside hard real-time alarm path unless strict real-time virtualisation is proven; monitors should remain on dedicated devices. VMs can be suitable for workstations, database replicas and imaging services when capacity and fail-over are designed properly.

TASK 12 – Re-evaluate and Adapt the Design

ASSIGNMENT QUESTION

New requirement: home remote-monitoring devices send readings over the internet and patient records move to a cloud service shared with partner hospitals. Analyse impact, evaluate original design, revise or redesign it, and defend the decision.

AIM

To understand re-evaluate and adapt the design and explain the related operating-system concepts clearly, using the given problem and examples.

EXPLANATION

TOPIC	EXPLANATION
Decision	MODIFY the original design, not replace it

	completely.
A. Impact	Home devices add unreliable internet links, remote authentication, device identity, secure updates and privacy requirements. Cloud records introduce distributed storage, service availability, encryption in transit and at rest, access federation, audit and data-residency considerations.
B. Evaluation	The original hard real-time monitor design remains suitable for local alarms because local alarms must not depend on internet or cloud availability. However, the original central on-premises database design must be extended to support cloud replication and secure partner access. Internet delay cannot guarantee a 10 ms remote alarm deadline; the device must detect urgent events locally.
C. Revised design	use an embedded real-time OS on home monitoring devices, local threshold checks and secure buffering. Send readings through authenticated encrypted connections to a cloud ingestion service. Use a distributed database with replication, backups, access controls and audit logs. Keep local hospital systems able to operate during cloud/network outages and synchronise later.
D. Defence	This hybrid approach preserves deterministic local response and adds scalability and cross-hospital sharing. Trade-offs include cloud costs, network dependence for remote viewing, added security administration and consistency delays between replicas. Store-and-forward buffering and tested disaster recovery reduce outage risk.